

As is, where is: does a tabular foundation model need the table cleaned first?

TabPFN-3.5 against 2013-style feature engineering on the Kaggle *Blue Book for Bulldozers* auction data

Agnaldo Calvi Benvenho*

October 2026

Abstract

Used machinery is auctioned *as is, where is*: in the condition it stands, without warranty or cleanup. We ask whether a tabular foundation model can price under the same clause, taking the data as published. TabPFN-3.5 receives the 2013 Kaggle *Blue Book for Bulldozers* auction table exactly as released, with the manufacturing year recorded as 1000 for 9.5% of the sales, the hour meter missing for 64% and zero for another 18%, 35 of 51 predictor columns more than half empty and the sale date as text. It is used zero-shot through the Prior Labs API and compared with a LightGBM given 2013-style cleaning and feature engineering. Every model sees the 401,125 sales through 2011 and predicts the 11,573 sales of January–April 2012. Five hypotheses and their refutation rules were committed before the first run. The raw table scores RMSLE 0.2200 against 0.2325 for the engineered baseline (paired bootstrap difference -0.0125 , 95% CI $[-0.0148, -0.0102]$), and its 80% predictive interval covers 82.0% of the realized prices. Cleaning or joining the “corrected” Machine Appendix improves TabPFN-3.5 by less than the pre-registered margin. One hypothesis is refuted: the most recent sales beat a random sample of equal size only for small contexts, and lose at 200,000 sales. On the same evaluation sales, the only published model we found, a tuned random forest, scores 0.2396.

Keywords: tabular foundation models; TabPFN; machinery and equipment valuation; auction prices; data cleaning; pre-registration; out-of-time evaluation.

1 Introduction

Valuing used heavy equipment from auction records is routine work for appraisers, lenders and insurers, and the records are dirty. Manufacturing years are missing or entered as placeholders, hour meters are blank or reset, configuration fields are filled for some sales and not others, and the same machine model appears under thousands of codes and free-text descriptions. In practice, a large share of the modelling effort goes into cleaning and engineering this table before any model is fitted.

Tabular foundation models change where that effort may be needed. TabPFN [5, 6] is pre-trained on synthetic tables and predicts a new table in context: the training rows are passed together with the rows to predict, and no model is fitted to the data at hand. If such a model can price from the table as published, the cleaning step stops being a prerequisite.

We test that claim on a canonical dirty benchmark, the 2013 Kaggle *Blue Book for Bulldozers* competition [7], under a design set against it. The bet is that TabPFN-3.5, with no cleaning and

*BP Avaliações e Perícias Ltda., São Paulo, Brazil. Mechanical engineer, machinery and equipment appraiser and court-appointed expert. Code and results: <https://github.com/abenvenho/tabpfn-bluebook-bulldozers> (Apache-2.0).

no tuning, prices the following months’ sales as well as a gradient-boosting model [4, 8] given 2013-style cleaning and feature engineering. Following a falsificationist reading [11], the bet is split into five hypotheses, each with a numeric rule that would refute it, committed to version control before the first TabPFN run.

The contributions are: (i) a pre-registered, out-of-time comparison of a zero-shot tabular foundation model with an engineered gradient-boosting baseline on raw auction data; (ii) a decomposition of the effect of cleaning, external corrected attributes, context size and context sampling; (iii) a reproducible pipeline whose API cost is quoted before each block and measured after it; and (iv) the findings, including one refutation.

2 Related work

The 2013 competition asked for the auction price of heavy equipment from usage, type and configuration. Published write-ups by top-ranked teams converge on the same recipe: separate models per product group or subgroup, blends of gradient boosting machines, random forests [1] and linear models, hyper-parameter search, and the removal of old or atypical periods [13, 9, 2]. The same teams report that the “corrected” Machine Appendix released by the organisers did not help or made results worse; one of them used it only to fill missing values. Parr and Howard [10] use the competition’s validation set as a test set for a tuned random forest, which provides the only published reference we found on the basis used here.

TabPFN was introduced as a transformer that solves small classification problems in a forward pass [5] and extended to regression [6]; later versions, including TabPFN-3.5, are served through the Prior Labs API [12]. Its claims are usually tested on curated benchmarks; this study asks what happens on a table that was never curated, against a baseline that was.

3 Data

3.1 Source and splits

The competition files contain 53 columns: the sale identifier, the price and 51 predictors. The split is by time, as published by the organisers (Table 1). The validation prices were released by Kaggle after the competition (`ValidSolution.csv`); the test prices never were. The raw files are not redistributed with the code; they are available at the competition page after accepting its rules.

Table 1: Data splits, as published by the competition.

Set	Period	Role here	Sales
Train	1989-01-17 to 2011-12-30	context / training	401,125
Valid	2012-01-01 to 2012-04-28	evaluation, prices from <code>ValidSolution.csv</code>	11,573
Test	2012-05-01 to 2012-11-16	submission file only; prices withheld	12,457

3.2 What is dirty

Table 2 groups the 51 predictors and reports what was measured on the training sales. In total, 35 of the 51 columns are more than half empty. Prices range from US\$4,750 to US\$142,000, with a median of US\$24,000. A separate Machine Appendix holds “corrected” attributes for 358,593 machines: manufacturing year, manufacturer and size class.

Table 2: Predictor groups and measured defects in the 401,125 training sales.

Group	Cols.	Defects
Machine and model identity	7	5,218 model codes; series empty in 86%, descriptor in 82%
Class and size	4	74 product classes described in text; size empty in 53%
Age and use	3	year = 1000 in 9.5%; hour meter empty in 64% and zero in 18%; usage band empty in 83%
The sale	4	date as text (3/14/2012 0:00); 53 state values; auctioneer empty in 5%
Configuration options	33	30 of 33 more than half empty; 26 also use the text “None or Unspecified”

4 Methods

4.1 Target and metric

All models predict $\log(1 + \text{price})$ and are scored with the competition’s metric,

$$\text{RMSLE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(1 + \hat{y}_i) - \log(1 + y_i))^2}, \quad (1)$$

which measures relative error. Predictions and quantiles are back-transformed with $\exp(\cdot) - 1$.

4.2 TabPFN-3.5

TabPFN-3.5 is called through the Prior Labs API (`tabpfn-client` 0.6.1) with default settings, zero-shot: no fitting to these data, no hyper-parameter search. Each run returns the mean prediction and the quantiles 0.10, 0.25, 0.50, 0.75 and 0.90; the 80% interval is [q10, q90]. Three versions of the table (*arms*) are compared:

- **raw** (primary): the 51 predictors exactly as in the CSV, including the sale date as text; nothing imputed or corrected;
- **clean**: 2013-style cleaning; years before 1900 and zero hour readings set to missing; the date replaced by year, month, day of week and days since 1989; age at sale added;
- **appendix**: the raw table plus six Machine Appendix attributes joined on the machine identifier.

The primary context is all 401,125 training sales. It was chosen by a rule committed in advance (the largest of all sales, 200k, 100k or 50k that fits the daily quota), applied to the server’s cost quote of 181,897 tokens per call before any result was seen. For the context hypothesis, contexts of 200k, 100k and 50k sales are drawn as the most recent sales or as a random sample (seed 42). Two further modes are explored outside the hypotheses: *fast* (the `v3.5-fast` model) and *thinking* (extra fit-time search, here optimising RMSE on the log scale; capped by the API at 200k rows, point predictions only).

4.3 Baselines

The **median** baseline predicts the training median (US\$24,000) for every sale. The **LightGBM** baseline [8] receives the clean arm’s cleaning and features (55 features), with text columns as native categorical features. Hyper-parameters are fixed: learning rate 0.05, at most 128 leaves, at least 20 sales per leaf, 80% of columns and rows sampled per tree, seed 42. The number of trees is chosen by early stopping (patience 200, at most 5,000) on the most recent 10% of

training sales (40,084 sales after 2010-10-30), with no 2012 data; the model is then refitted on all training sales with the selected 995 trees. Unlike the 2013 top teams, it uses no per-group models, no blending and no hyper-parameter search.

4.4 Protocol and hypotheses

Hypotheses, thresholds and analysis rules were committed to git before the first TabPFN run on the real data (Table 3). Paired comparisons use a bootstrap over the 11,573 validation sales [3]: 10,000 resamples with replacement, seed 42, percentile 95% interval of the RMSLE difference. A materiality margin of 0.005 RMSLE applies to H1 and H2. A script applies the rules and writes the verdicts; an independent AI agent, which had not taken part in producing the results, recomputed every number in the repository’s README with its own code.

Table 3: Pre-registered hypotheses and refutation rules.

	Hypothesis	Refuted if
H1	Raw TabPFN-3.5 is not worse than the engineered LightGBM	worse by > 0.005 RMSLE with 95% CI above 0
H2	Cleaning, or joining the Machine Appendix, does not improve TabPFN-3.5	improvement > 0.005 with CI above 0
H3	At equal context size, the most recent sales beat a random sample	$\text{RMSLE}(\text{recent}) - \text{RMSLE}(\text{random}) \geq 0$ at any size
H4	The 80% interval holds four months ahead	coverage outside 75–85%
H5	The primary run scores in the 2013 top 10% on the leaderboard-mirror split	$\text{RMSLE} > 0.25339$

5 Results

5.1 Verdicts

Table 4 gives the pre-registered verdicts and Figure 1 every configuration. Four hypotheses survive; H3 is refuted.

Table 4: Pre-registered verdicts on the January–April 2012 sales. Differences are RMSLE of the first model minus the second; negative means the first errs less.

	Difference	Value	95% CI	Verdict
H1	raw – LightGBM	−0.01245	[−0.01480, −0.01021]	corroborated
H2	raw – clean	+0.00175	[+0.00083, +0.00270]	corroborated (below margin)
H2	raw – appendix	+0.00267	[+0.00150, +0.00388]	corroborated (below margin)
H3	recent – random, 50k	−0.00639	[−0.00839, −0.00440]	supports
H3	recent – random, 100k	−0.00032	[−0.00215, +0.00152]	supports by the rule; tie
H3	recent – random, 200k	+0.00234	[+0.00079, +0.00389]	refutes
H4	80% interval coverage	82.0%	band 75–85%	corroborated
H5	primary RMSLE	0.22001	threshold 0.25339	corroborated, with caveat

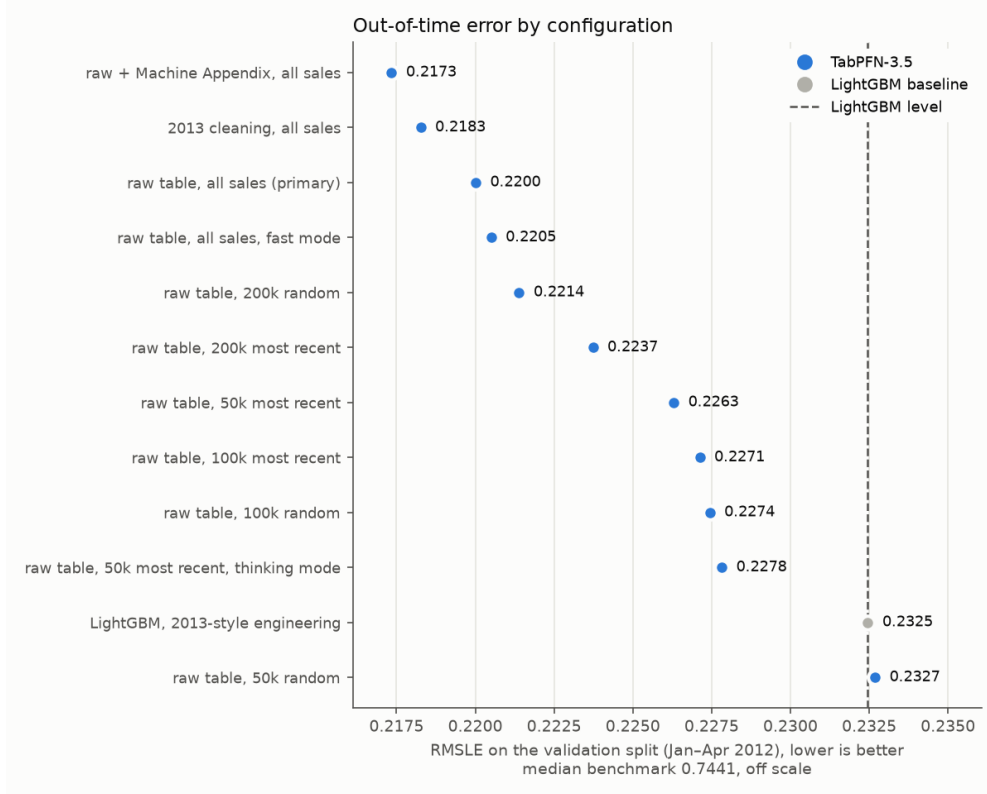


Figure 1: Out-of-time RMSLE of each distinct configuration. The dashed line is the engineered LightGBM; the median benchmark (0.7441) is off scale.

H1. The pre-registered claim was the weak one, “not worse”. The raw table is better, with the whole interval below zero: RMSLE 0.22001 against 0.23246. In a point comparison outside the pre-registration, ten of the eleven distinct TabPFN-3.5 configurations score below the baseline; the exception is a random sample of 50k sales (0.23268).

H2. The clean arm scores 0.21826 and the appendix arm 0.21734, the best configuration overall. Both improvements are real (intervals above zero) but below the 0.005 margin, so H2 survives by the frozen rule; read without the margin, “cleaning does not improve” would be false. The gains are about a seventh (cleaning) and a fifth (appendix) of the advantage the raw table already holds over the baseline. The 2013 finding that the appendix did not help [13, 9, 2] is not reproduced in its strong form: with TabPFN-3.5 it helps a little.

5.2 Context size and recency (H3)

H3 was stated as a universal claim over context sizes, and the 200k comparison refutes it with an interval entirely above zero (Figure 2). With 50k sales the most recent ones clearly win; at 100k the two samplings tie; at 200k a random sample across 23 years of auctions beats the most recent 200k. The two curves meet at all 401,125 sales, the best point by point estimate. For an appraiser the pattern reads like the trade-off in choosing comparables: with few, recency carries the current market; with many, coverage of models and configurations matters more. That reading was not itself tested.

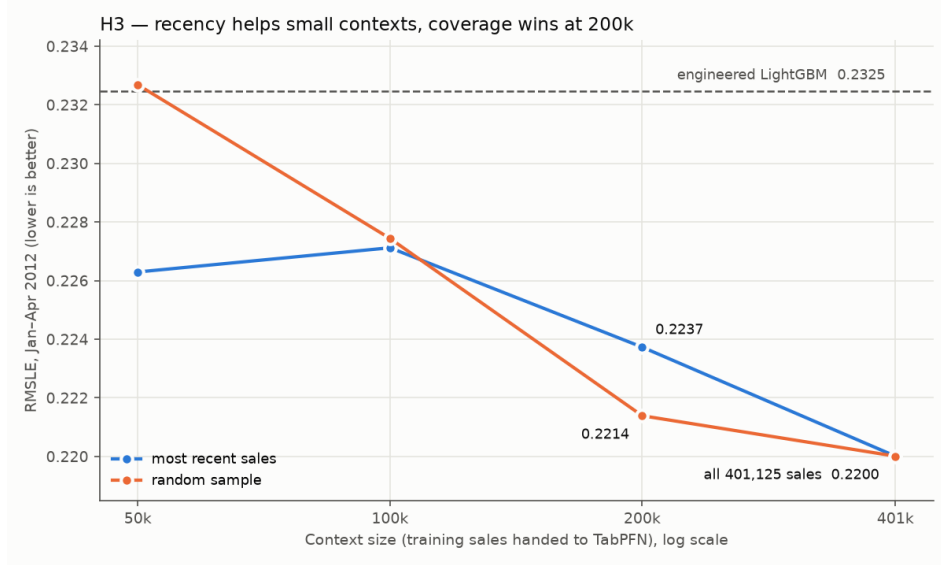


Figure 2: RMSLE by context size for the most recent sales and for a random sample of equal size.

5.3 Predictive interval (H4)

The 80% interval covers 82.0% of the realized prices four months ahead. All five quantiles sit slightly below the diagonal (Figure 3): 7%, 19%, 44%, 71% and 89% of realized prices fall at or below q10, q25, q50, q75 and q90. Realized prices ran 2.6% above the predictions in geometric mean (mean log residual +0.0256). The cause was not tested. A general price rise is not supported, since the January–April median sale price was US\$27,000 in both 2011 and 2012; seasonality is compatible, since in 2009, 2010 and 2011 the January–April median exceeded the full-year median (23,000 vs 22,000; 24,000 vs 23,500; 27,000 vs 26,000).

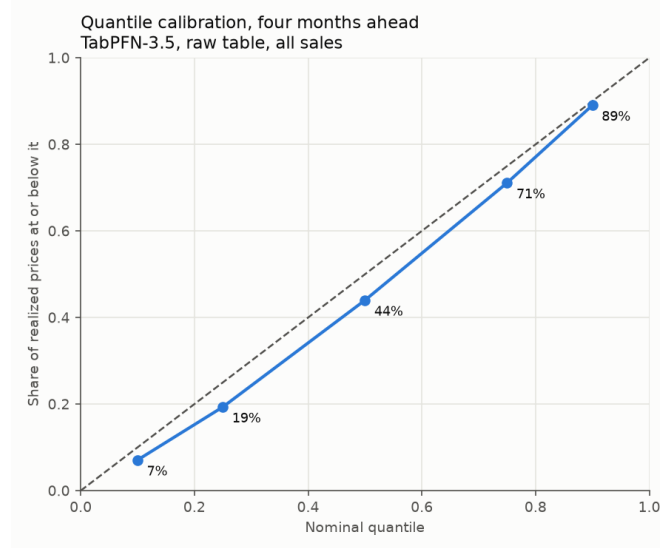


Figure 3: Quantile calibration of the primary run: share of realized prices at or below each predicted quantile.

5.4 Leaderboard and an external reference (H5)

The primary RMSLE of 0.22001 is below the 2013 top-10% mark (0.25339) and numerically below the winning score (0.22909). Those scores were computed by Kaggle on the hidden May–November 2012 test set, by models trained with data through April 2012. Kaggle no longer

accepts late submissions, so the comparison is indicative, as pre-registered, and no rank is claimed. The public leaderboard was computed on the same January–April 2012 sales as here, but it is unusable: after the validation prices were released, 108 of 475 teams show an RMSLE of 0.0 on it [10].

Parr and Howard [10] report the only model we found on the same basis: Kaggle’s validation set as test set, training on sales through 2011 (from 2007 on). Their tuned random forest, with feature selection and an inflation adjustment, scores 0.2396 (Table 5). Their predictions are not available, so the comparison is by point estimate only.

Table 5: Models evaluated on the January–April 2012 sales and trained on sales through 2011.

Model	RMSLE
TabPFN-3.5, raw table + Machine Appendix, all sales	0.21734
TabPFN-3.5, raw table, all sales (primary)	0.22001
LightGBM, 2013-style cleaning and features (this study)	0.23246
Random forest, Parr and Howard [10]	0.2396
Median benchmark	0.74414

5.5 Exploratory results (not pre-registered)

Fast mode. With all sales, **v3.5-fast** scores 0.22050, a difference of +0.00049 (95% CI $[-0.00065, +0.00163]$) against the base model, in 813 s of wall time instead of 1,654 s.

Thinking mode. It failed on the server twice at the planned 200k context and ran at 50k, scoring 0.22782 against 0.22630 for the base model on the same context (difference +0.00152, CI $[+0.00033, +0.00274]$). Without a time column, which the API requires as datetimes or numbers while the raw arm keeps the date as text, its internal validation does not use contiguous blocks of time; whether that explains the result was not tested.

Reproducibility. The same 50k configuration, run on two different days, returned identical means and quantiles for all 11,573 sales.

Cost. Usage was read from the API’s counter after each block on 2026-10-01. The server quote matched the measured spend exactly for the arms (732,938 tokens) and context (326,828 tokens) blocks; the Kaggle submission block cost twice its quote (730,546 against 365,272), consistent with an automatic client retry but not confirmed. The blocks of that day used 2,357,700 tokens, within a daily limit of 5,000,000.

6 Discussion

For valuation practice the main result is that the cleaning step stopped being a prerequisite on this table. A zero-shot model given the auction records as they were published priced the next four months better than a gradient-boosting model given 2013-style cleaning and features, and it delivered predictive intervals that held their nominal coverage. Cleaning and corrected attributes still helped, but by a small amount relative to the advantage already obtained without them. This matters where the data come from many sellers and are never cleaned to a common standard, which is the usual case for equipment auction records.

The refuted hypothesis is as informative as the corroborated ones. A practitioner’s instinct, to prefer recent sales, held only when the context was small; with large contexts, breadth won. The small upward bias of the realized prices, with an interval that still held, is a reminder that a model conditioned on past sales can lag the season or market it predicts; here its cause was not established.

Limits are clear. There is one evaluation period, January to April 2012. The baseline is plain: fixed hyper-parameters, no Machine Appendix, no per-group models and no blending, all of which the 2013 top teams used, so a stronger adversary is possible. There is one table and

one market. The hidden test set could not be scored. Deviations from the pre-registration are listed in Appendix A.

7 Conclusion

Given the *Blue Book for Bulldozers* table as it was published, TabPFN-3.5 used zero-shot predicted the next four months’ auction prices with RMSLE 0.2200, against 0.2325 for a LightGBM with 2013-style cleaning and features and 0.2396 for a published tuned random forest on the same evaluation sales. Four of five pre-registered hypotheses survived; the claim that recent sales are always the better context was refuted at 200,000 sales. Code, results, the pre-registration and the full history of the runs are public.

Data and code availability

Code, pre-registration, predictions, metrics, verdicts and figures: <https://github.com/abenvanho/tabpfn-bluebook-bulldozers> (Apache-2.0). Data: Kaggle *Blue Book for Bulldozers*, <https://www.kaggle.com/c/bluebook-for-bulldozers>, not redistributed. A synthetic smoke test runs the full pipeline with no API access.

References

- [1] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [2] Dataiku. Kaggle contest: Blue Book for Bulldozers. Blog post, 26 April 2013. <https://blog.dataiku.com/2013/04/26/kaggle-contest-blue-book-for-bulldozers>.
- [3] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, New York, 1993.
- [4] J. H. Friedman. Greedy function approximation: a gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.
- [5] N. Hollmann, S. Müller, K. Eggenberger, and F. Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. In *International Conference on Learning Representations (ICLR)*, 2023.
- [6] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637:319–326, 2025. doi:10.1038/s41586-024-08328-6.
- [7] Kaggle and Fast Iron. Blue Book for Bulldozers. Competition, 2013. <https://www.kaggle.com/c/bluebook-for-bulldozers>.
- [8] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, pages 3146–3154, 2017.
- [9] A. Olariu. Trees, ridges and bulldozers made in 1000 AD. Blog post, 2013. <http://webmining.oliariu.org/trees-ridges-and-bulldozers-made-in-1000-ad/>.
- [10] T. Parr and J. Howard. *The Mechanics of Machine Learning*, chapter “Bulldozer testing”. Online book. <https://mlbook.explained.ai/bulldozer-testing.html>.
- [11] K. R. Popper. *The Logic of Scientific Discovery*. Hutchinson, London, 1959.
- [12] Prior Labs. TabPFN (software) and TabPFN-3.5 model card. <https://github.com/PriorLabs/TabPFN>; https://huggingface.co/Prior-Labs/tabpfn_3_5.
- [13] Y. Seroussi. Fitting noise: forecasting the sale price of bulldozers (Kaggle competition summary). Blog post, 19 November 2014. <https://yanirseroussi.com/2014/11/19/fitting-noise-forecasting-the-sale-price-of-bulldozers-kaggle-competition-summary/>.

A Deviations from the pre-registration

1. **H5 referee.** Kaggle late submissions turned out to be closed; H5 was rewritten before any run to the leaderboard-mirror comparison.
2. **Pilot before the context freeze.** A 50k pilot ran before the primary context was frozen. The selection rule was mechanical and already committed, so the pilot could not influence it.
3. **Primary context.** The rule selected all sales, from the server quote recorded in the pre-registration; a later re-quote gave the same values.
4. **Modes.** Thinking was moved from the primary context to 200k before any modes run, because of the API cap; it then failed twice server-side and ran at 50k. The fast and thinking model identifiers in the runner were corrected after the H2 arms had run; the base-model path used by every hypothesis was not changed.
5. **H3 at 100k.** The rule uses the point estimate (-0.00032), which counts as supporting, although the interval spans zero. H3 is refuted by the 200k comparison either way.
6. **H3 rule wording.** The pre-registration writes the rule as “random $\geq$ recent (diff ≤ 0)” without defining *diff*. It was read as in the hypothesis table and the verdict script, both committed before any run. H3 is refuted under either reading.
7. **Unexplained quota use on 2026-09-30.** The daily quota ran out after the pilot and primary runs, whose quotes add up to about 0.4M tokens; usage was not yet being read, so the rest cannot be attributed.

B Settings

Item	Value
TabPFN client	<code>tabpf-client</code> 0.6.1, default settings, server default model (v3.5)
Quantiles	0.10, 0.25, 0.50, 0.75, 0.90
Cost quote per call (pricing quota_v3)	all sales 181,897; 200k 53,211; 100k 18,496; 50k 10,000 tokens
LightGBM	4.7.0; learning rate 0.05; 128 leaves; min. 20 per leaf; 0.8 column/row sampling
LightGBM trees	early stopping, patience 200, max. 5,000; selected 995; 55 features
Bootstrap	10,000 paired resamples, seed 42, percentile 95% interval
Environment	Python 3.11+, versions pinned in <code>requirements.txt</code>